

1. Adaptive Filtering: Least Mean Squares (LMS)

Use-cases include:

- System identification
- Noise cancellation
- Prediction

Wiener Filtering

Wiener filters: a class of linear optimum discrete-time filters:

$$y(n) = \sum_{k=0}^{\infty} w_k^* u(n-k)$$

Where $u(n)$ and $d(n)$ are **WSS with zero mean**.

The goal is to design a filter that minimises the MSE

$$J = \mathbb{E}[e(n)e^*(n)] = \mathbb{E}[|e(n)|^2] = \mathbb{E}[|d(n) - y(n)|^2]$$

Wiener optimal solution:

$$\nabla_k J = \nabla_k \mathbb{E}[e(n)e^*(n)] = -2\mathbb{E}[u(n-k)e^*(n)]$$

$$\nabla_k J = \mathbb{E}[u(n-k)e_o^*(n)] = 0$$

Principle of orthogonality: when the filter operates optimally (i.e., J is minimum), $e_o(n)$ is orthogonal to each input sample $u(n-k)$ contributing to the estimation of the desired response.

Wiener-Hopf equations

From the Wiener optimal solution, we can expand by inserting e_o^* :

$$\mathbb{E} \left[u(n-k) \left(d^*(n) - \sum_{i=0}^{\infty} w_{oi} u^*(n-i) \right) \right] = 0$$

$$\sum_{i=0}^{\infty} w_{oi} \mathbb{E}[u(n-k)u^*(n-i)] = \mathbb{E}[u(n-k)d^*(n)]$$

This leads to the Wiener-Hopf equations

$$\sum_{i=0}^{\infty} w_{oi} r(i-k) = p(-k)$$

With:

$$\mathbb{E}[u(n-k)u^*(n-i)] = r(i-k)$$

$$\mathbb{E}[u(n-k)d^*(n)] = p(-k)$$

Matrix representation:

$$\mathbf{R} \mathbf{w}_o = \mathbf{p}$$

With:

$$\mathbf{R} = \mathbb{E}[\mathbf{u}(n)\mathbf{u}^H(n)] \text{ (Correlation Matrix)}$$

$$\mathbf{w}_o = (w_{o0}, w_{o1}, \dots, w_{o,m-1})^T \text{ (Optimal weights)}$$

$$\mathbf{p} = \mathbb{E}[\mathbf{u}(n)d^*(n)] = [p(0), p(-1), \dots, p(M-1)]^T$$

(Cross-correlation vector between $u(n)$ and $d^*(n)$)

NOTICE: $\mathbf{R}$ increases in size and inverting is difficult

Steepest descent

With SD the statistics are *known*:

→ Gradient and filter weights are deterministic

→ Optimal, as it reaches Wiener Solution as $n \rightarrow \infty$

On Wiener-filtering (update rule):

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu[\mathbf{p} - \mathbf{R}\mathbf{w}(n)]$$

Stochastic Gradient Descent

With the SGD the statistics are *unknown*:

→ Easier implementation (no inverse $\mathbf{R}$)

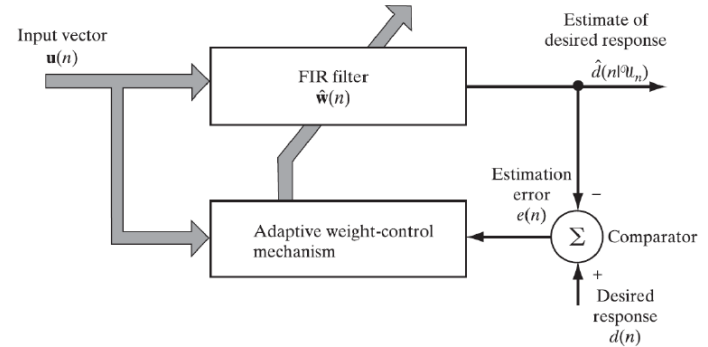
→ Suboptimal, as it will randomly walk around the Wiener Solution (no equilibrium)

→ *Linear law of scaling* w.r.t adjustable parameters

LMS Implementation using SGD

- Simple w.r.t. computational complexity and dim.
- Does *not* require knowledge of statistical characteristics
- *Robust* w.r.t noise and interference.
- Weights approaches the Wiener solution for $n \rightarrow \infty$

Overall block diagram of an (LMS) adaptive filter:



Adaptation based on the instantaneous error signal $e(n)$ → Stochastic gradient descent: we **get rid of the expectation operator**.

Scalar implementation:

$$J_s(n) = e(n)e^*(n) = |e(n)|^2$$

$$\nabla_{J_s, k}(n) = \frac{\partial J_s(n)}{\partial w_k^*} = -2u(n-k)e^*(n)$$

LMS updating rule:

$$\hat{w}_k(n+1) = \hat{w}_k(n) - \frac{1}{2} \mu \nabla_{J_s, k}(n)$$

$$\hat{w}_k(n+1) = \hat{w}_k(n) + \mu \cdot u(n-k)e^*(n)$$

LMS vector implementation:

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu \cdot \mathbf{u}(n)e^*(n)$$

Stability criterion for μ (robustness):

$$0 < \mu < \frac{2}{\lambda_{max}}$$

NOTICE: Instantaneous estimates for $\mathbf{R}$ and $\mathbf{p}$ are included by the feedback and *averaged over time*.

Normalized LMS

Same as LMS, but with a normalization factor of $\|\mathbf{u}(n)\|^2$

We dislike large steps in weights →

The factor comes from the minimization of the updating rule:

$$\|\delta \hat{\mathbf{w}}(n+1)\|^2 = \|\hat{\mathbf{w}}(n+1) - \hat{\mathbf{w}}(n)\|^2 \text{ subject to}$$

$$\hat{\mathbf{w}}^H(n+1)\mathbf{u}(n) = d(n)$$

Through derivatives of the cost function w.r.t the weights, the NLMS updating rule is finally formulated as:

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \frac{\mu}{\delta + \|\mathbf{u}(n)\|^2} \cdot \mathbf{u}(n-k)e^*(n)$$

Where the δ is included for numerical stability.

NMLS potentially converges faster than LMS.

2. Adaptive Filtering: Recursive Least Squares (RLS)

Least-squares Filter:

Takes the same offset as for Wiener filtering (same diagram), but ε is minimised:

$$\varepsilon = \sum_{i=M}^N e(i)e^*(i) = \sum_{i=M}^N |e(i)|^2$$

$$\nabla_k \varepsilon = \sum_{i=M}^N u(i-k)e_{min}^*(i) = 0$$

Principle of orthogonality: when the FIR filter operates optimally (i.e., in its least-squares condition), the minimum-error time series $e_{min}(i)$ is orthogonal to the time series $u(i-k)$

Normal Equations:

The normal equations are an alternative to the principle of orthogonality which describes the least-squares condition of a FIR filter.

$$\sum_{i=M}^N u(i-k)e_{min}^*(i) = 0, \quad \text{and}$$

$$e_{min}(i) = d(i) - \sum_{t=0}^{M-1} \hat{w}_t^* u(i-t)$$

This gives the normal equations:

$$\sum_{t=0}^{M-1} \hat{w}_t \phi(t, k) = \varphi(-k)$$

With:

$$\sum_{i=M}^N u(i-k)u^*(i-t) = \phi(t, k) \quad (\text{Correlation})$$

$$\sum_{i=M}^N u(i-k)d^*(i) = \varphi(-k) \quad (\text{Cross-correlation})$$

Matrix representation:

$$\Phi \hat{\mathbf{w}} = \boldsymbol{\varphi}$$

With:

$$\Phi = \begin{bmatrix} \phi(0,0) & \phi(1,0) & \dots & \phi(M-1,0) \\ \phi(0,1) & \phi(1,1) & \dots & \phi(M-1,1) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(0,M-1) & \phi(1,M-1) & \dots & \phi(M-1,M-1) \end{bmatrix}$$

$$\hat{\mathbf{w}} = (\hat{w}_0, \hat{w}_1, \dots, \hat{w}_{M-1})^T$$

$$\boldsymbol{\varphi} = [\varphi(0), \varphi(-1), \dots, \varphi(M-1)]^T$$

The default LMS filter has issues w.r.t. the number of samples affecting the filter coefficients. **This is bad if $u(n)$ or $d(n)$ are non-stationary.** To solve this weighted least squares can be applied, where a weight matrix $\mathbf{B}$ is introduced so (new data is weighted greater than old data):

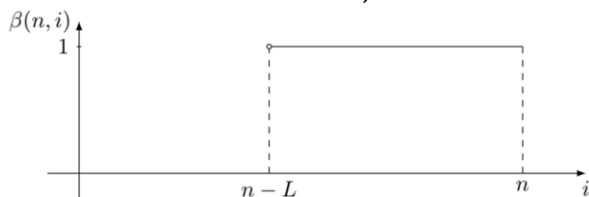
$$J(\mathbf{w}(n)) = \mathbf{e}^T(n)\mathbf{B}(n)\mathbf{e}(n)$$

With: $\mathbf{B}(n) = \text{diag}(\beta(n,1), \beta(n,2), \dots, \beta(n,n))$

Two weight-functions are common “sliding window weight function” and “exponential weight function”.

Sliding window weight function

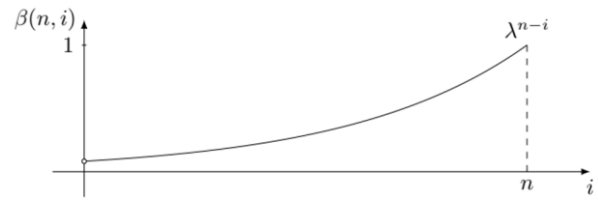
$$\beta(n, i) = \begin{cases} 1, & n-L < i \leq n \\ 0, & \text{otherwise} \end{cases}$$



Exponential weight function

$$\beta(n, i) = \begin{cases} \lambda^{n-i}, & 0 < i \leq n \\ 0, & \text{otherwise} \end{cases}$$

Where λ is the forgetting factor



This makes the output less dependent on old data, and therefore better performing.

**** Måske effektiv længde ****

Recursive least-squares

To solve the weighted normal equations:

$$\Phi(n)\mathbf{w}(n) = \boldsymbol{\varphi}(n)$$

One must solve:

$$\mathbf{w}(n) = \Phi^{-1}(n)\boldsymbol{\varphi}(n)$$

Which yields a **high computational complexity**. Instead, recursive methods for calculating *correlation matrix* Φ and *cross correlation vector* $\boldsymbol{\varphi}$ is wanted. These can, using **exponential weight**, be calculated as (example):

$$\Phi(n) = \mathbf{u}(n)\mathbf{u}^T(n) + \lambda\Phi(n-1)$$

$$\boldsymbol{\varphi}(n) = \mathbf{u}(n)d(n) + \lambda\boldsymbol{\varphi}(n-1)$$

However here the inverse of the correlation matrix must still be calculated. Using the matrix inversion lemma the inverse can be expressed as:

$$\mathbf{P}(n) = \lambda^{-1}(\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{u}^T(n)\mathbf{P}(n-1))$$

Where:

$$\mathbf{P}(n) = \Phi^{-1}(n) \quad (\text{inverse correlation matrix})$$

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{u}(n)}{\lambda^{-1}\mathbf{u}^T(n)\mathbf{P}(n-1)\mathbf{u}(n)}, \quad (\text{gain vector})$$

From this the filter coefficients can be calculated recursively as:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n)$$

Where the priori error is:

$$\xi(n) = d(n) - \mathbf{u}^T(n)\mathbf{w}(n-1)$$

From this the RLS algorithm can be formulated as:

$$\boldsymbol{\pi}(n) = \mathbf{P}(n-1)\mathbf{u}(n)$$

$$\mathbf{k}(n) = \frac{\boldsymbol{\pi}(n)}{\lambda\mathbf{u}^T(n)\boldsymbol{\pi}(n)}$$

$$\xi = d(n) - \mathbf{u}^T(n)\mathbf{w}(n-1)$$

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n)$$

$$\mathbf{P}(n) = \lambda^{-1}(\mathbf{P}(n-1) - \mathbf{k}(n)\boldsymbol{\pi}^T(n))$$

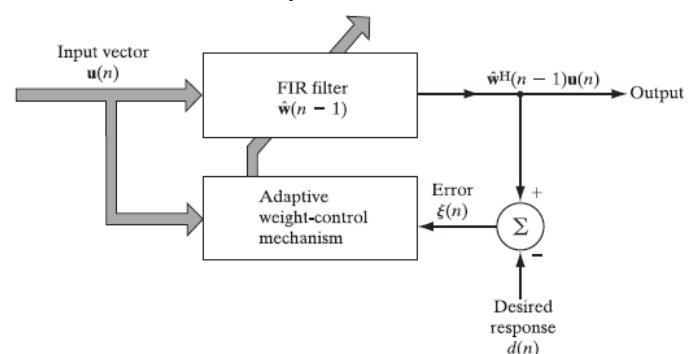
With the $O(M^2)$ for an M-tap FIR filter, but with faster convergence.

The initialization variables for the biased case are:

$$\mathbf{P}(0) = \delta^{-1}\mathbf{I} \quad (\delta \text{ is a small value})$$

$$\mathbf{w}(0) = \mathbf{0}$$

$$u(n) = 0, \quad \text{for } -M+1 < n < 1$$



3. Multidimensional Fourier Transform

Fourier Series:

Assume function f defined over finite interval T :

$$f(t) = \sum_{k=-\infty}^{+\infty} a_k e^{i2\pi k \frac{t}{T}}, \quad a_k = \frac{1}{T} \int_T f(t) e^{-i2\pi k \frac{t}{T}} dt$$

Fourier Transform (Analysis equation)

Given a function $f \in L^1(\mathbb{R})$ we define the Fourier transform $\mathcal{F}$ as:

$$F(k) = \mathcal{F}\{f(t)\} = \int_{\mathbb{R}} f(t) e^{-i2\pi kt} dt$$

Inverse Fourier Transform (Synthesis equation)

Given a function $F \in L^1(\mathbb{R})$ we define the inverse Fourier transform $\mathcal{F}^{-1}$ as:

$$\hat{f}(t) = \mathcal{F}^{-1}\{F(k)\} = \int_{\mathbb{R}} F(k) e^{i2\pi kt} dk$$

ANY PERIODIC FUNCTION CAN BE REPRESENTED EXACTLY USING FOURIER SERIES

General properties of Fourier

Spectrum: $S_f(k) = |F(k)|^2$, $F(k) = \mathcal{F}\{f(t)\}$

Bandwidth: $F(k) = 0 \quad \forall k \notin B$, $F(k) = \mathcal{F}\{f(t)\}$

Duality property

The Fourier transforms are related by the following relation

$$\left. \begin{aligned} \mathcal{F}\{f\} &= \mathcal{F}^{-1}\{f^{-}\} \\ \mathcal{F}\{\mathcal{F}\{f\}\} &= f^{-} \end{aligned} \right\}, \quad \text{where} \quad f^{-}(x) = f(-x)$$

Parity of a function:

$f(x)$ is even if $f(-x) = f(x)$

$f(x)$ is odd if $f(-x) = -f(x)$

Fourier Transform follows the following properties:

Theorem	$f(x)$	$F(k)$
Similarity	$f(ax)$	$\frac{1}{ a } F\left(\frac{k}{a}\right)$
Addition	$f(x) + g(x)$	$F(k) + G(k)$
Homogeneity	$af(x)$	$aF(k)$
Time/phase shift	$f(x - a)$	$e^{-i2\pi ak} F(k)$
Convolution	$f(x) * g(x)$	$F(k)G(k)$
Derivative	$f'(x)$	$i2\pi k \cdot F(k)$

Rayleigh's Theorem (Energy):

$$\int_{-\infty}^{\infty} |f(x)|^2 dx = \int_{-\infty}^{\infty} |F(k)|^2 dk$$

Parseval's Theorem (Power):

$$\int_{-\infty}^{\infty} f(x) g^*(x) dx = \int_{-\infty}^{\infty} F(k) G^*(k) dk$$

Multidimensional Fourier transform

Consider the signal $f(\mathbf{x})$ with $\mathbf{x} = [x_1, x_2, \dots, x_n] \in \mathbb{R}^n$

Fourier transform:

$$F(\xi) = \mathcal{F}\{f(\mathbf{x})\} = \int_{\mathbb{R}^n} f(\mathbf{r}) e^{-i2\pi \mathbf{x} \cdot \xi} d\mathbf{x}$$

Inverse Fourier transform:

$$\hat{f}(\mathbf{x}) = \mathcal{F}^{-1}\{F(\xi)\} = \int_{\mathbb{R}^n} F(\xi) e^{-i2\pi \xi \cdot \mathbf{x}} d\xi$$

Note that the exponential function is determined by the product of the vectors $\mathbf{x}$ and ξ , where $\xi = [\xi_1, \xi_2, \dots, \xi_n]$ is the *frequency vector*.

$$\mathbf{x} \cdot \xi = x_1 \xi_1 + x_2 \xi_2 + \dots + x_n \xi_n$$

Why have a vector of frequencies?

For a fixed ξ , the previous equations lead to a family of parallel lines in the (x_1, x_2) -plane (or in the spatial domain).

Where the vector ξ is normal to each one of the lines. Where the distance between these two consecutive lines is just given by

$$\text{distance} = \frac{1}{\sqrt{\xi_1^2 + \xi_2^2}} = \frac{1}{\|\xi\|}$$

Periodicity

Using the previous results, for a fixed ξ , we can parameterize the function using the frequency normal vector as a reference. Given any starting point $\mathbf{a} = (a_1, a_2)$ we obtain that:

$$\mathbf{x}(t) = \mathbf{a} + t \frac{\xi}{\|\xi\|}$$

Therefore, the complex exponential function adopts the following form:

$$e^{\pm i2\pi \mathbf{x} \cdot \xi} = e^{\pm i2\pi \mathbf{a} \cdot \xi} \cdot e^{\pm i2\pi t \|\xi\|}$$

where the first term is constant and does not depend on t . However, the second factor is periodic of period $1/\|\xi\|$. Therefore, although the frequency now is a vector, ξ , we can keep most of the terminology for the one-dimensional case.

Separability:

If a multivariable function can be written as a product of single-value functions:

$$\begin{aligned} f(x_1, x_2, \dots, x_n) &= f_1(x_1) f_2(x_2) \dots f_n(x_n) \\ \mathcal{F}\{f(x_1, x_2, \dots, x_n)\} &= \mathcal{F}\{f_1(x_1)\} \mathcal{F}\{f_2(x_2)\} \dots \mathcal{F}\{f_n(x_n)\} \end{aligned}$$

Theorem	$f(\mathbf{x})$	$F(\xi)$
Similarity	$f(\mathbf{ax})$ with $\mathbf{a} \in \mathbb{R}^n$	$\frac{1}{ a_1 a_2 \dots a_n } F\left(\frac{\xi_1}{a_1}, \frac{\xi_2}{a_2}, \dots, \frac{\xi_n}{a_n}\right)$
Addition	$f(\mathbf{x}) + g(\mathbf{x})$	$F(\xi) + G(\xi)$
Shift	$f(\mathbf{x} - \mathbf{a})$	$e^{-i2\pi \mathbf{a} \cdot \xi} F(\xi)$
Conv.	$f(\mathbf{x}) * g(\mathbf{x})$	$F(\xi) G(\xi)$
Deriv.	$\frac{\partial}{\partial x_i} f(\mathbf{x})$	$i2\pi \xi_i \cdot F(\xi)$

Rayleigh's Theorem (Energy):

$$\int_{\mathbb{R}^n} |f(\mathbf{x})|^2 d\mathbf{x} = \int_{\mathbb{R}^n} |F(\xi)|^2 d\xi$$

Parseval's Theorem (Power):

$$\int_{\mathbb{R}^n} f(\mathbf{x}) g^*(\mathbf{x}) d\mathbf{x} = \int_{\mathbb{R}^n} F(\xi) G^*(\xi) d\xi$$

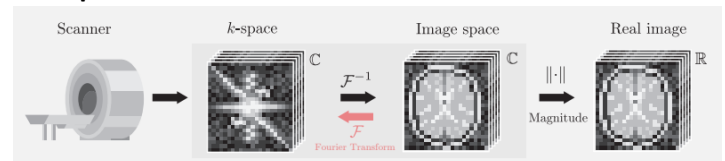
General Stretch Theorem:

For any linear transformation, $\mathbf{A}$, of the domain of the function, its corresponding Fourier transform is given by:

$$\mathcal{F}\{f(\mathbf{Ax})\} = \frac{1}{|\det \mathbf{A}|} F\left((\mathbf{A}^{-1})^T \xi\right)$$

A particular case of the general stretch theorem is rotations: any two-dimensional signal, any rotation $\mathbf{R}$ over the space domains produce exactly the same rotation in the frequency domain. Duality holds again, and parity is the same.

Example:



**** Indsæt billede af hjerne ****

4. Detection theory/Hypothesis testing

How do we make a decision?

- Daily life $\rightarrow$ no specific ruleset
- Sometimes a particular rule or method

Engineers:

- Formulate decision rule
- Optimality criterion

The goal of detection theory is to *select one of K mutually exclusive hypotheses*, $\mathcal{H} = \{h_0, \dots, h_{K-1}\}$, based on some observations, $\mathbf{x} = [x_1, \dots, x_n]^T$. $\mathbf{X}$ is used as a realization of a random vector with pdf f_X .

Probability Model for Decision Problems

To do this a decision rule is used, which is a function $\hat{H}(\mathbf{x})$, that assigns a hypothesis for each possible observation.

$$\hat{H}: \text{range}(\mathbf{X}) \rightarrow \mathcal{H}$$

We may have some *prior knowledge* about the *likelihood* of a hypothesis being true: P_k is the **prior probability** of the hypothesis, where $\sum_{k=0}^{K-1} P_k = 1$.

After making the observations, our view of the probability of a given hypothesis being true changes. Now we say

$$\mathbb{P}(\hat{H} \text{ is true given } \mathbf{x}) = \mathbb{P}(H = h_k | \mathbf{x})$$

Also known as the *a posteriori* probability of h_k being true.

Where: $f_{X|H}(\mathbf{x}|h_k)$ is the **likelihood** for h_k and is a conditional pdf of $\mathbf{X}$. $\mathbf{X}$ depends on which of the hypotheses are true $\rightarrow$ *conditional pdf of $\mathbf{X}$ under hypothesis h_k (likelihood for h_k)*, $f_{X|H}(\mathbf{x}|h_k)$.

Bayes theorem can expand the *a posteriori* probability and include the *prior* and *likelihood*:

$$\mathbb{P}(H = h_k | \mathbf{x}) = \frac{f_{X|H}(\mathbf{x}|h_k)P_k}{f_X(\mathbf{x})}$$

The denominator will be disregarded since it's the same regardless of h_k and will not enter the decision rules when derived. If we consider the decision rule evaluated at the random data $\mathbf{X}$, then the decision $\hat{H}(\mathbf{X})$ is a random variable.

Maximum A posterior probability (MAP) Decision rule

The MAP decision rule picks the hypothesis which has the highest probability of being correct. That is:

$$\hat{H}_{Map}(\mathbf{x}) = \arg \max_{h_k \in \mathcal{H}} f_{X|H}(\mathbf{x}|h_k)P_k$$

The *prior* probability affects the probability significantly. $P_k = 0$ means the hypothesis will never be selected, and the opposite if $P_k = 1$. Furthermore, *the more probable a hypothesis is (before we observe the data), the more convincing the data needs to be in order for us to change our minds.*

Decision regions: the regions of the $\mathbf{x}$ -axis for which the rule will select a respective hypothesis

Maximum likelihood (ML) Decision rule

In cases where **no priori probability** is available or undesirable to use, one can instead set all priori probabilities equal. This results in the ML-decision rule:

$$\hat{H}_{ML}(\mathbf{x}) = \arg \max_{h_k \in \mathcal{H}} f_{X|H}(\mathbf{x}|h_k)$$

Bayes' decision rule

Instead of maximising a probability, **Bayes' decision rule minimises a cost/risk of choosing a particular hypothesis**. For this a cost $C_{kk'}$ is assigned for if h_k is decided and $h_{k'}$ is the true. Diagonal cost should be low, since these correspond to "true" decisions.

	$H = h_0$	$H = h_1$	...
$\hat{H} = h_0$	C_{00}	C_{01}	
$\hat{H} = h_1$	C_{10}	C_{11}	
$\vdots$			

The cost/risk of a decision conditioned on $\mathbf{x}$ is formulated as:

$$C(h_k | \mathbf{x}) = \sum_{k'=0}^{K-1} C_{kk'} \mathbb{P}(H = h_{k'} | \mathbf{x})$$

The decision rule then chooses the hypothesis with the lowest expected cost:

$$\hat{H}_{Bayes}(\mathbf{x}) = \arg \min_{h_k \in \mathcal{H}} C(h_k | \mathbf{x})$$

Thresholds for the Binary case

The binary decision rule can be written as:

$$\hat{H} = \begin{cases} h_0, & \text{if } \Lambda(\mathbf{x}) \geq \gamma \\ h_1 & \text{otherwise} \end{cases}, \quad \Lambda(\mathbf{x}) = \frac{f_{X|H}(\mathbf{x}|H_0)}{f_{X|H}(\mathbf{x}|H_1)}$$

The *threshold* γ is **1 for ML**, **P_1/P_0 for MAP**, and for Bayes:

$$\gamma_{Bayes} = \frac{(C_{11} - C_{01})P_1}{(C_{00} - C_{10})P_0}$$

Other decision rules

MAP, ML, and Bayes are not the only decision rules.

Two prominent examples of other decision rules are:

- The Neyman-Pearson rule
- Mini-max rule

5. Minimum mean squared error

MMSE is optimal among all possible estimators for θ , in the sense that it minimizes the mean squared error of the parameter, it is an estimator that is difficult to apply. We need to compute a conditional expectation of the unknown parameter given the data, for which we need a full statistical description of the dependency. Joint pdf of the data and unknown parameter.

MMSE Estimator

Why estimation?

- Provide solutions that work in general (variety of environments), adaptive to different cases
- The estimation of unknown values found in the environment, is a crucial task

Examples:

- User position for GPS
- The direction of arrival for sound signals on an array of microphones
- The speed of a ship navigating through the ocean, controlled by automatic control system

Estimation Problem Formulation:

Given a set of data $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, estimate the value of an unknown quantity θ .

What is an estimator? (Definition)

An estimator is a function $\hat{\theta} = g(\mathbf{x})$ of data $\mathbf{x}$ used to 'guess' the value of some unknown entity.

Estimation error definitions

- Estimation error: $\hat{\theta} - \theta$ (one-shot)
- Bias (mean error): $\mathbb{E}[\hat{\theta} - \theta]$ (tendency)
- Mean Squared error: $\mathbb{E}[(\hat{\theta} - \theta)^2]$ (circumvents positive and negative estimation errors cancelling out)

MMSE

The MMSE of θ given the data $\mathbf{x}$ is the function $\hat{\theta} = g(\mathbf{x})$ such that the MSE $\mathbb{E}[(\hat{\theta} - \theta)^2]$ is minimum. From this we define the MMSE estimator as:

$$\hat{\theta}_{MMSE}(\mathbf{X}) = \arg \min_{\hat{\theta}} \mathbb{E}[(\hat{\theta} - \theta)^2]$$

Using the law of total expectation, we get:

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathbb{E}[\mathbb{E}[(\hat{\theta} - \theta)^2 | \mathbf{X}]]$$

From this we can find the minimum by differentiating and equating to 0.

$$\frac{\partial \mathbb{E}[\mathbb{E}[(\hat{\theta} - \theta)^2 | \mathbf{X}]]}{\partial \hat{\theta}} = 2\hat{\theta} - 2\mathbb{E}[\theta | \mathbf{X}] = 0$$

$$\hat{\theta}_{MMSE} = \mathbb{E}[\theta | \mathbf{x}]$$

Properties of MMSE:

- Unbiased: $\mathbb{E}[\theta - \hat{\theta}] = \mathbb{E}[\theta - \mathbb{E}_x[\theta | \mathbf{x}]] = \mathbb{E}[\theta] - \mathbb{E}[\theta] = 0$
- Lowest MSE among all estimators, by definition
- Fulfills the "orthogonality principle", the error is uncorrelated with any function of the data

Remarks to the MMSE:

- Simple to derive, but needs full statistical description:

$$\mathbb{E}[\theta | \mathbf{x}] = \int \theta \underbrace{p(\theta | \mathbf{x})}_{\text{posterior}} d\theta = g(\mathbf{x})$$
- Analytical solutions are not always available
- $\mathbb{E}_x[\theta | \mathbf{x}]$, may be computed through Monte Carlo sims.

Conditional expectation: $\mathbb{E}[\theta | \mathbf{x}] = \int \theta \underbrace{p(\theta | \mathbf{x})}_{\text{posterior}} d\theta$

We can use Bayes' Rule:

$$p(\theta | \mathbf{x}) = \frac{p(\theta, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\theta, \mathbf{x})}{\int p(\theta, \mathbf{x}) d\theta}$$

In order to compute the MMSE we need the posterior, and in order to obtain then we need the joint pdf of θ and $\mathbf{x}$ (full statistical description, no good).

Linear MMSE estimator

- Keeps some good aspect
- Much simpler

How do we simplify? Constrain the functional form that the estimators can take

Properties of LMMSE:

- Only first and second order statistical modeling is necessary (**Mean of theta, mean of data, covariance matrix of the data, and the covariance between the data and theta**)
- Computational complexity of the estimator is fixed and low, N product and additions (from vector multiplication), for every realization. Covariance does not change "on the fly"
- LMMSE estimator is unbiased
- Minimises all MSE estimators among all affine estimators.
- Fulfills orthogonality principle for affine functions of data $\mathbf{X}$

To solve the problems of MMSE, the estimation function can be constrained to a linear form:

$$\hat{\theta}(\mathbf{x}) = h_0 + \sum_{n=1}^N h_n x_n = h_0 + \mathbf{h}^T \mathbf{x}$$

This changes the problem to:

$$\hat{\theta}_{LMMSE}(\mathbf{X}) = \arg \min_{h_0, \mathbf{h}} E[(h_0 + \mathbf{h}^T \mathbf{x} - \theta)^2]$$

Solving this problem is rather simple given: $h_0 = E[\theta] - \mathbf{h}^T E[\mathbf{x}]$ and $\mathbf{h} = \mathbf{C}_{xx}^{-1} \mathbf{C}_{x\theta}$, ($\mathbf{C}_{x\theta}^T = \mathbf{C}_{\theta x}$):

$$\hat{\theta}_{LMMSE}(\mathbf{X}) = E[\theta] + \mathbf{C}_{\theta x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Where: $\mathbf{C}_{\theta x} = E[(\mathbf{x} - E[\mathbf{x}])(\theta - E[\theta])^T]$, (covariance vector)
 $\mathbf{C}_{xx} = E[(\mathbf{x} - E[\mathbf{x}])(\mathbf{x} - E[\mathbf{x}])^T]$. (covariance matrix)

This means that to compute we only need $E[\theta]$, $E[\mathbf{x}]$, $\mathbf{C}_{xx}$, $\mathbf{C}_{\theta x}$. The LMMSE is unbiased, and fulfils a version of the orthogonality principle, where we constrain the function $h(\mathbf{x})$ to be linear.

The MSE of the LMMSE can also be directly calculated as:

$$MSE_{LMMSE} = E[(\hat{\theta}(\mathbf{x}) - \theta)^2] = \text{var}(\theta) - \mathbf{C}_{\theta x} \mathbf{C}_{xx}^{-1} \mathbf{C}_{x\theta}$$

So the MSE of the LMMSE is available before implementation.

LMMSE for multiple variables

To extend for multiple variables, θ becomes $\boldsymbol{\theta} = [\theta_1, \dots, \theta_K]^T$, which means the error we want to minimise becomes:

$$MSE_{\hat{\boldsymbol{\theta}}} = \sum_{k=1}^K MSE_{\hat{\theta}_k} = \sum_{k=1}^K E[(\hat{\theta}_k(\mathbf{x}) - \theta_k)^2]$$

The estimated variables can here be calculated as:

$$\hat{\theta}_k = E[\theta_k] + \mathbf{C}_{\theta_k x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Each variable k can be but as a row in a matrix, leading to the matrix notation:

$$\hat{\boldsymbol{\theta}} = E[\boldsymbol{\theta}] + \mathbf{C}_{\boldsymbol{\theta} x} \mathbf{C}_{xx}^{-1} (\mathbf{x} - E[\mathbf{x}])$$

Where $\mathbf{C}_{\boldsymbol{\theta} x} = E[(\boldsymbol{\theta} - E[\boldsymbol{\theta}])(\mathbf{x} - E[\mathbf{x}])^T]$

The total MSE can be calculated directly as the sum of MSEs from the single case:

$$MSE_{\hat{\boldsymbol{\theta}}} = \sum_K MSE_{\hat{\theta}_k} = \text{Tr}\{\mathbf{C}_{\boldsymbol{\theta} \boldsymbol{\theta}} - \mathbf{C}_{\boldsymbol{\theta} x} \mathbf{C}_{xx}^{-1} \mathbf{C}_{x \boldsymbol{\theta}}\}$$

The trace is used as the error is on the diagonals of the matrices.

6. Kalman filter

Why do we need a recursive LMMSE Estimator?

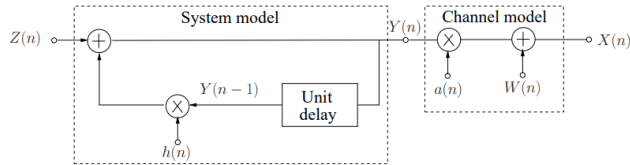
- Covariance matrix of data increases in size for every new data. And the number of constants increases.
- We do not like inverse covariance matrix, which would be necessary.
- Can we find a more efficient implementation of the estimates $\hat{y}(n)$ for all $n = 1, 2, \dots$?

Recursive LMMSE Estimator: $\hat{y}(n+1) =$

linear function $\{\hat{y}(n), x(n+1)\}$ (**Kalman Filter**).

Goal: To find LMMSE estimates of the samples of an **unobservable process** $Y(n)$ from the samples of a **related, observable process** $X(n)$, for $n = 1, 2, \dots$

Recursive implementation: at time $n+1$, the LMMSE estimate of $Y(n+1)$ is obtained as a linear function of the LMMSE estimate of $Y(n)$ and the new observation $X(n+1)$



State equation: $Y(n) = h(n)Y(n-1) + Z(n)$, $n = 1, 2, \dots$

Channel equation: $X(n) = a(n)Y(n) + W(n)$, $n = 1, 2, \dots$

Where:

- $h(n)$ and $a(n)$ are known, deterministic sequences
- $Z(n)$ and $W(n)$ are white noise processes with zero mean, and variances $\sigma_Z^2(n)$ and $\sigma_W^2(n)$
- $Y(0)$ is a r.v. with known mean $\mu_{Y(0)}$ and variance $\sigma_{Y(0)}^2$
- Assume $Y(n)$, $Z(n)$, and $W(n)$ mutually uncorrelated

System and Channel Model shows how:

- Each sample of $Y(n)$ is related to the previous sample
- How the observable $X(n)$ is related to $Y(n)$ through a weight and a summation with a noise process. Aka. Each of the samples of $X(n)$ is a weighted and noisy version of the unobservable $Y(n)$.

We want to find a **recursive implementation** of the LMMSEE of $Y(n)$ based on observations $X(1), \dots, X(n)$. We define (where the hat denotes LMMSEE):

$\hat{Y}(n|n)$ **Estimation of $Y(n)$**

$\hat{Y}(n+1|n)$ **One-step prediction of $Y(n+1)$ at time n**

$\hat{X}(n+1|n)$ **One-step prediction of $X(n+1)$**

And the recursive implementation:

$$\underbrace{\hat{Y}(n+1|n+1)}_{\text{Estimation at time } n+1} \equiv \mathcal{LF}\left\{ \underbrace{\hat{Y}(n|n)}_{\text{Estimation at time } n}, \underbrace{X(n+1)}_{\text{Observation at time } n+1} \right\}$$

Such a recursive implementation of the LMMSEE exists. It is called the **Kalman Filter**.

The recursion is a two-step process: One-step prediction and updating.

The MSE $\mathbb{E}\left[\left(Y(n) - \hat{Y}(n|n)\right)^2\right] = R(n|n)$ can also be computed recursively.

Prediction step:

$$\hat{Y}(n+1|n) = h(n+1)\hat{Y}(n|n)$$

$$R(n+1|n) = h(n+1)^2 R(n|n) + \sigma_Z^2(n+1) \quad (\text{MSE})$$

$$\hat{X}(n+1|n) = a(n+1)\hat{Y}(n+1|n)$$

$\hat{Y}(n+1|n)$ satisfies the orthogonality principle and is therefore the LMMSEE.

Updating step:

$$b(n+1) \equiv \frac{a(n+1)R(n+1|n)}{a(n+1)^2 R(n+1|n) + \sigma_W^2(n+1)} \quad (\text{the Kalman gain})$$

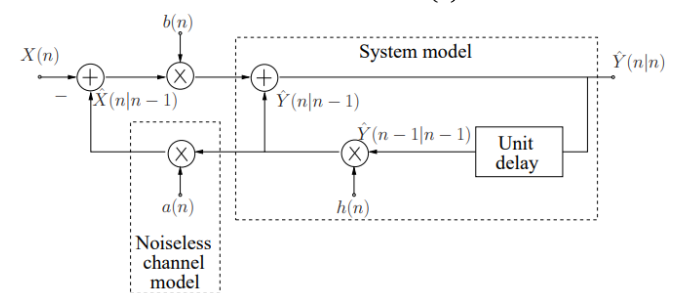
$\hat{Y}(n+1|n+1) = \hat{Y}(n+1|n) + b(n+1)[X(n+1) - \hat{X}(n+1|n)]$ LMMSEE of $Y(n+1)$, based on the prediction AND a correction factor. Satisfies Orthogonality Principle $\rightarrow$ LMMSEE.

$$R(n+1|n+1) = [1 - b(n+1)a(n+1)]R(n+1|n)$$

Initialization:

$$\hat{Y}(0|0) = \mu_{Y(0)}$$

$$R(0|0) = \sigma_{Y(0)}^2$$



Vector version

When extending to a vector version, the models become:

$$\mathbf{y}(n) = \mathbf{H}(n)\mathbf{y}(n-1) + \mathbf{z}(n), \quad n = 1, 2, \dots$$

$$\mathbf{x}(n) = \mathbf{A}(n)\mathbf{y}(n) + \mathbf{w}(n), \quad n = 1, 2, \dots$$

$\mathbf{H}(n)$ and $\mathbf{A}(n)$ are now known matrices describing the relation between previous estimates and new. $\mathbf{z}(n)$ and $\mathbf{w}(n)$ is now vectors of noise. The steps of the Kalman filter changes to:

Prediction Step:

$$\hat{\mathbf{y}}(n|n-1) = \mathbf{H}(n)\hat{\mathbf{y}}(n-1|n-1)$$

$$\hat{\mathbf{x}}(n|n-1) = \mathbf{A}(n)\hat{\mathbf{y}}(n|n-1)$$

$$\mathbf{R}(n|n-1) = \mathbf{H}(n)\mathbf{R}(n-1|n-1)\mathbf{H}(n)^T + \mathbf{Q}_z(n)$$

Updating Step:

$$\hat{\mathbf{y}}(n|n) = \hat{\mathbf{y}}(n|n-1) + \mathbf{B}(n)(\mathbf{x}(n) - \hat{\mathbf{x}}(n|n-1))$$

$$\mathbf{R}(n|n) = (\mathbf{I} - \mathbf{B}(n)\mathbf{A}(n))\mathbf{R}(n|n-1)$$

$$\mathbf{B}(n) = \mathbf{R}(n|n-1)\mathbf{A}(n)^T (\mathbf{A}(n)\mathbf{R}(n|n-1)\mathbf{A}(n)^T + \mathbf{Q}_w(n))^{-1}$$

Initialization:

$$\mathbf{y}(0) = \mu_{\mathbf{y}(0)}$$

$$\mathbf{R}(0|0) = \text{cov}(\mathbf{y}(0))$$

SENSOR-FUSION

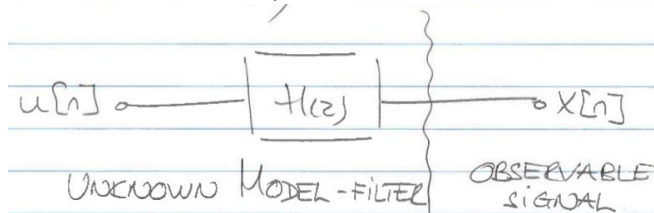
7. Linear predictive coding: general overview

Summary:

- establish the Normal Equations from an observable signal (LTI)

how to, which is assumed being generated from a model LTI-system having only poles, i.e., an AR-filter (Yule-Walker equations). pick out a segment of the observable signal using a window function, leading to the fact that the segment, being used for calculating $\mathbf{R}$ and $\mathbf{r}$, is identically equal to zero outside this specific time-interval. This technique is known as the Auto-correlation Method because the elements in $\mathbf{R}$ and $\mathbf{r}$ are auto-correlation coefficients with different lags. Finally using Levinson-Durbin method is used to circumvent the need for inverting the autocorrelation for every sample.

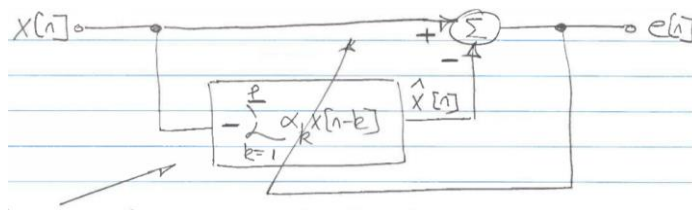
Assumes that the observable signal $x[n]$ is generated by an LTI system $H(z)$ excited by a white signal $u[n]$. This means that $x[n]$ contains all information about $H(z)$, and thereby $|H(e^{j\omega})|$ represents the Spectral content. In general, the filter can be expressed as an ARMA-model:



$$H(z) = \frac{X(z)}{U(z)} = G \cdot \frac{1 + \sum_{l=1}^q b_l z^{-l}}{1 + \sum_{k=1}^p a_k z^{-k}}$$

Then by estimating b_l, a_k and G from $x[n]$ for a given q and p , then we have conducted a model-based spectral analysis. If we consider the above as parameters, this method is known as "Parametric Spectral Analysis".

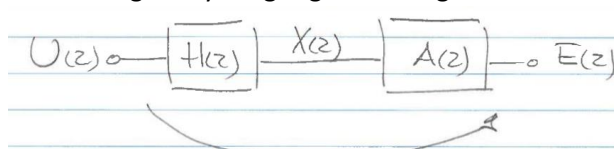
Working with the AR case (a_k). From here the idea is that the observable signal can be predicted using a **one-step linear predictor** based on previous samples. This can be represented as:



The goal now is to **minimize the error**; $e[n] = x[n] - \hat{x}[n]$, where $\hat{x}[n] = -\sum_{k=1}^p \alpha_k x[n-k]$. **Transforming this to the z-domain**, we can find the inverse filter $A(z)$ as:

$$\frac{E(z)}{X(z)} = A(z) = \frac{1}{H(z)} = 1 + \sum_{k=1}^p \alpha_k z^{-k}$$

Connecting everything together we get:



$$\frac{E(z)}{U(z)} = H(z)A(z) = G \cdot \frac{1}{1 + \sum_{k=1}^p a_k z^{-k}} \cdot \left(1 + \sum_{k=1}^p \alpha_k z^{-k}\right) = G$$

Since the goal is to find $\alpha_k = a_k$. This leads to the relation between the inverse filter and the LTI system as:

$$A(z) = \frac{G}{H(z)}$$

For an entire input sequence and p -order filter, this can be represented in matrix form as:

$$\mathbf{X}\mathbf{a} - \mathbf{b} = \mathbf{e}$$

DISCLAIMER: $\mathbf{a}$ is composed of α 's (alpha)

From here we try to minimize the energy in the predicted error, also known as Error-variance.

$$E = \mathbf{e}^T \mathbf{e} = \mathbf{a}^T \mathbf{X}^T \mathbf{X} \mathbf{a} - 2\mathbf{a}^T \mathbf{X}^T \mathbf{b} + \mathbf{b}^T \mathbf{b}$$

From this, it can be seen that, the error-variance is a quadratic function of $\mathbf{a}$ (convex). The error-variance can now be minimized in two ways "Differentiate wrt. $\mathbf{a}$ and set = 0" or using "the orthogonal principle". Both methods lead to the normal equations:

$$\mathbf{X}^T \mathbf{X} \mathbf{a} = \mathbf{X}^T \mathbf{b}$$

From this some interpretations can be made. First $\mathbf{X}^T \mathbf{X} = \mathbf{R}$ is a symmetric Toeplitz autocorrelation matrix, and second $\mathbf{X}^T \mathbf{b} = -\mathbf{r}$ is an autocorrelation vector. This reduces the normal equations to the Yule-Walker equations: $\mathbf{R}\mathbf{a} = -\mathbf{r}$.

The minimum error variance, also known as **residual variance**, is: $E_{min} = R(0) + \sum_{k=1}^p \alpha_k R(k)$

To calculate the gain:

It's assumed that the energy in the signals is identical; $E[\hat{x}^2[n]] = E[x^2[n]]$. This means that from the residual variance the gain can be calculated as: $G = \sqrt{E_{min}}$.

Efficiency

The Yule-Walker equations can be expensive to calculate, since the autocorrelation matrix has to be inverted. Instead, a method to recursively solve the equations for increasing order without the inverse is wanted. To this the **Levinson-Durbin algorithm** is used.

$$E_0 = r(0)$$

for $m = 1$ to p

$$k_m = -\frac{r(m) + \sum_{k=1}^{m-1} a_{m-1,k} \cdot r(m-k)}{E_{m-1}}$$

$$a_{mm} = k_m$$

for $k = 1$ to $m-1$

$$a_{m,k} = a_{m-1,k} + k_m \cdot a_{m-1,m-k}$$

end

$$E_m = E_{m-1} \cdot (1 - k_m^2)$$

end

NOTICE: k_m are applied as *dummy variables*, but they are useful for other stuff (Reflection/Parcor Coefficients)

Using the Levinson-Durbin Algorithm for solving a p th order system, all LPC coefficients for lower order systems are also calculated!

8. Linear predictive coding: filter design by modelling

Looking instead at the frequency domain, we can define the **residual error** as:

$$E_p = \frac{1}{2\pi} \int_{-\pi}^{\pi} |E(e^{j\omega})|^2 d\omega = \frac{1}{2\pi} \int_{-\pi}^{\pi} A(e^{j\omega}) A(e^{-j\omega}) P(\omega) d\omega$$

Where $P(\omega)$ is the power spectrum of $x[n]$. Differentiating as $\frac{\partial E_p}{\partial a_i} = 0$, for $0 \leq i \leq p$ results in:

$$\frac{\partial E_p}{\partial a_i} = 0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} 2 \begin{bmatrix} \cos(\omega) \\ \cos(2\omega) \\ \vdots \\ \cos(p\omega) \end{bmatrix} + 2 \begin{bmatrix} 1 & \cos(\omega) & \dots & \cos((p-1)\omega) \\ \cos(\omega) & 1 & \dots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ \cos((p-1)\omega) & \dots & \dots & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} P(\omega) d\omega$$

Looking at the power spectrum we find: $P(\omega) = |X(e^{j\omega})|^2 = X(e^{j\omega})X(e^{-j\omega})$ taking the IDTFT of this results in: $p[n] = R(n)$. From this, a relation can be formulated:

$$R(n) \xleftrightarrow{DTFT} P(\omega)$$

From this it can be shown that the **power spectrum is Real and Even**, which results in:

$$R(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} P(\omega) \cos(\omega) d\omega$$

Comparing this with the integral $\frac{\partial E_p}{\partial a_i}$ above it can be expressed in terms of the autocorrelation as:

$$2 \begin{bmatrix} R(1) \\ R(2) \\ \vdots \\ R(p) \end{bmatrix} + 2 \begin{bmatrix} R(0) & R(1) & \dots & R(p-1) \\ R(1) & R(0) & \dots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ R(p-1) & \dots & \dots & R(0) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = 0$$

Which can be reformulated to the Yule-Walker equations: **$Ra = -r$** .

Power spectrum

From the model, a formulation of the power spectrum can be formed given $u[n]$ is white and $|X(e^{j\omega})|^2 = P(\omega)$:

$$P(\omega) = G^2 |H(e^{j\omega})|^2$$

$$\hat{P}(\omega) = \frac{G^2}{|A(e^{j\omega})|^2}$$

Looking at the entire system, a relation can be formulated as:

$$\frac{|E(e^{j\omega})|^2}{P(\omega)} = \frac{G^2}{\hat{P}(\omega)}$$

Which means that for the model to have the same power spectrum $|E(e^{j\omega})|^2 = G^2(\text{constant})$, which happens when $e[n]$ is white. This means that $A(z)$ must be a whitening filter.

$$\hat{P}(\omega) = \frac{G^2}{|A(e^{j\omega})|^2} = \frac{G^2}{|1 + \sum_{k=1}^p a_k e^{j\omega k}|^2}$$

We can then conclude that any power spectrum can be approximated by an AR-model of a sufficiently large model order p .

From the desired amplitude spectrum to power spectrum

1. Design spectrum $|H(e^{-j\omega})|$
2. Extend spectrum to 2π
3. Convert to power spectrum $|H(e^{j\omega})|^2$
4. Evaluate autocorrelation coefficients $R(n) = \text{IDTFT}\{|H(e^{j\omega})|^2\}$
5. Create **R** and **$-r$**
6. Use either AR- or MA-modelling, depending on dips or peaks.

AR-modelling

Given the relation between the autocorrelation and power spectrum: $R(n) \xleftrightarrow{DTFT} P(\omega)$, knowing the **desired amplitude spectrum** $|H(e^{j\omega})|$ we can derive the **power spectrum** $|H(e^{j\omega})|^2$ and then calculate the autocorrelation coefficients as:

$$R(n) = \text{IDTFT}(|H(e^{j\omega})|^2)$$

This autocorrelation function can then be used in the Yule-Walker equations to calculate the filter coefficients:

$$Ra = -r \Rightarrow A(z)$$

Which then can be used to calculate the expected power spectrum:

$$\hat{P}(\omega) = \left| \frac{G}{A(e^{j\omega})} \right|^2 = \frac{G^2}{|1 + \sum_{k=1}^p a_k e^{-jk\omega}|^2}$$

NB: is worse at hitting dips than spikes (only consists of poles)

MA-modelling

The idea for MA-modelling is to first find a high-order AR-model, and then use that as the impulse response for the model filter. First solve the Yule-Walker from the AR with a high order p , to find **$a_L[n]$** . Then use this to calculate the autocorrelation of the MA-model for order q .

$$R_a[k] = \sum_{n=0}^L a[n]a[n+k], \quad 0 \leq k \leq L$$

This autocorrelation is then used to calculate the Yule-Walker equations:

$$R_a b = -r_a$$

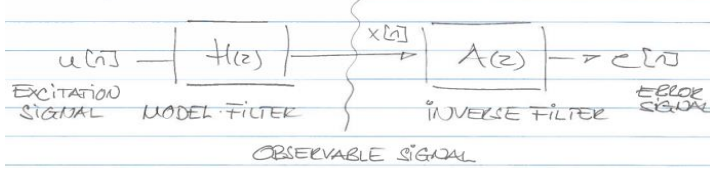
The results here can then be used in the formulation of the MA as:

$$B(z) = \sum_{k=0}^M b_k z^{-k}, \quad b_0 = 1$$

NB: is worse at hitting spikes than dips (only consists of zeros)

9. Linear predictive coding: lattice filter

OUR SCENARIO:



We wish to determine the inverse filter $A(z)$ of a given order p .

This is done by LMS optimization on the residual signal $e[n]$.

The **lattice filter** is an alternative structure for the filter $A(z)$ which is derived from the LD-algorithm. From the LD-algorithm

$a_k^{(m)}$ is found as:

$$a_k^{(m)} = a_k^{(m-1)} - k_m a_{m-k}^{(m-1)}$$

Using this in the definition of $A(z)$:

$$A^{(m)}(z) = 1 + \sum_{k=1}^m a_k^{(m)} z^{-k}$$

Inserting the formulation from above:

$$\begin{aligned} A^{(m)}(z) &= 1 + \sum_{k=1}^m (a_k^{(m-1)} - k_m a_{m-k}^{(m-1)}) z^{-k} \\ &= A^{(m-1)}(z) - k_m z^{-m} A^{(m-1)}(z^{-1}) \end{aligned}$$

With the forward-error of the system $E^{(m)}(z) = A^{(m)}(z)X(z)$, the above equation can be inserted, resulting in:

$$E^{(m)}(z) = \underbrace{A^{(m-1)}(z)X(z)}_{(m-1)^{\text{st}} \text{ order inverse filter}} - k_m \underbrace{z^{-m} A^{(m-1)}(z^{-1})X(z)}_{\text{Let's look at this term}}$$

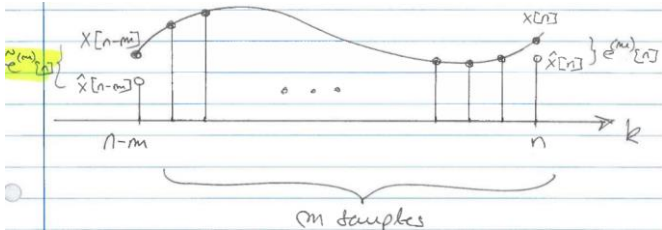
We define a new polynomial

$$\tilde{E}^{(m)}(z) \triangleq z^{-m} A^{(m)}(z^{-1})X(z)$$

Transforming to time-domain we find:

$$\tilde{e}^{(m)}[n] = x[n-m] + \underbrace{\sum_{k=1}^m a_k^{(m)} x[n-m+k]}_{\text{Backward Prediction Error}}$$

Notice the summation over m "increases backwards $-1, -2, \dots$ ": this means that $\tilde{e}^{(m)}[n]$ is a linear combination of delayed samples of $x[n]$. This can be interpreted as the **backwards prediction error**.



Changing the formulation slightly so it's describing the previous backwards error.

$$z^{-1} \tilde{E}^{(m-1)}(z) = z^{-m} A^{(m-1)}(z^{-1})X(z)$$

which when inserted in the definition of the current forward error gives:

$$E^{(m)}(z) = E^{(m-1)}(z) - k_m z^{-1} \tilde{E}^{(m-1)}(z)$$

We now want to find an equation describing the backwards prediction error based on previous errors. From the formulation of $A(z)$ and the definition of the $\tilde{E}^{(m)}(z)$, we get:

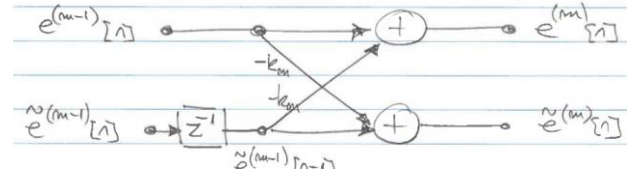
$$\begin{aligned} \tilde{E}^{(m)}(z) &= z^{-m} (A^{(m-1)}(z^{-1}) - k_m z^m A^{(m-1)}(z)) X(z) \\ &= z^{-1} \tilde{E}^{(m-1)}(z) - k_m E^{(m-1)}(z) \end{aligned}$$

Taking the two expressions for the errors and converting to time domain we find:

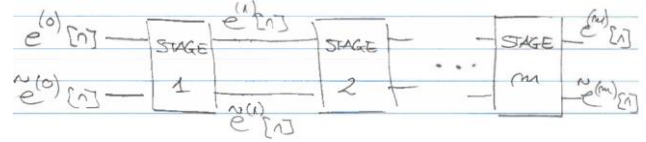
$$\begin{aligned} e^{(m)}[n] &= e^{(m-1)}[n] - k_m \tilde{e}^{(m-1)}[n-1] \\ \tilde{e}^{(m)}[n] &= \tilde{e}^{(m-1)}[n-1] - k_m e^{(m-1)}[n] \end{aligned}$$

This set of difference equations describes the evolution of the error based on the previous ones (m^{th} order forward and backward prediction errors). From this we can conclude that **instead of the**

ordinary tapped-delay-line FIR-filter $A(z)$ we can implement it as a cascade of m sections of:



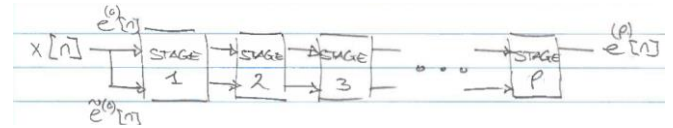
OBS The above graph corresponds to one iteration of the Levinson-Durbin Algorithm.



It can be seen that the input to the filter is different however, so what does the 0th error mean? The error is calculated from the difference of the true and predicted input $x[n]$, but if $m = 0$, there is no prediction, hence:

$$e^{(0)}[n] = \tilde{e}^{(0)}[n] = x[n]$$

p^{th} order FIR lattice filter



p^{th} order IIR lattice filter

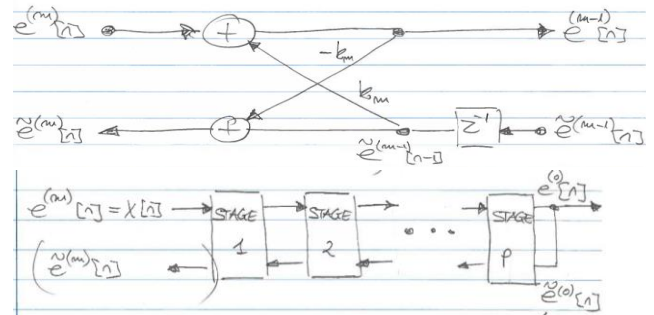
In order to design an IIR lattice filter, we maintain the lattice structure, but interchange the input and output

$$A^{(m)}(z) = \frac{E^{(0)}(z)}{E^{(m)}(z)}$$

Which is the inverse of the previous, and so

$$A^{(m)}(z) = \frac{1}{1 + \sum_{k=1}^m a_k^{(m)} z^{-k}}$$

Identical to the AR Model Filter



FIR coefficients

The coefficients are calculated from the residual signals(error), and for order m becomes:

$$k_m = - \frac{\sum_{n=0}^{N-1} (e^{(m-1)}[n] \cdot \tilde{e}^{(m-1)}[n-1])}{\sum_{n=0}^{N-1} (\tilde{e}^{(m-1)}[n])^2}$$

NB: Should be numerical smaller than zero.

NOTE:

- i^{th} filter coefficient is calculated based on residual signals from the $(i-1)^{\text{st}}$ lattice stage
- The forward and backward Residual variances are identical for order m
- Compared to FIR implementation of LPC, we do not use the columns from the Levinson-Durbin algorithm, but the diagonal (k_m) ensuring values between -1 and 1
- This is good for fixed-point calculations (proof omitted)